Lecture 5.3: Managing Data for Reproducibility: Git I

AREC 570 / ECON 530 / Unit 5

Lauren Chenarides

Reading and sequence

Gentzkow and Shapiro, *Code and Data for the Social Sciences*, Sections 1-4
(Gentzkow and Shapiro 2014)

1. Connect the API exercise to a reproducible project
2. Organize files so the project can run on another computer
3. Define the roles of Git, GitHub, and RStudio
4. Choose among creating, forking, templating, and cloning a repository
5. Practice the pull, edit, stage, commit, and push workflow

Where we left off in Lecture 5.2

The `arec_570_demo` project retrieved land-use, streamflow, weather, and Census data through APIs

- R scripts recorded the requests
- Raw downloads preserved the responses
- A retrieval log recorded source, date, and parameters
- `docs/data-plan.md` connected the sources to a research question

Today we organize that work and begin recording its history

Follow-along repository

`arec_570_demo` develops in stages across Unit 5

Tag	Lecture	Addition
`5.2-retrieve`	API collection	Retrieval script
`5.3-structure`	Structure and Git	Folders, `run_all.R`, and `.gitignore`
`5.4-collaborate`	Collaboration	Issues, branches, and pull requests
`5.5-report`	Reporting	Codebook, sample, and data section

Reproducibility starts with organization

Can another researcher reconstruct a result from the same inputs?

They need

- Original inputs
- Code for every transformation
- The order in which the code runs
- Software and package requirements
- The project version used for the result

Source: Gentzkow and Shapiro (2014)

Start with a recognizable project root

```
arec_570_demo/
  README.md
  arec_570_demo.Rproj
  run_all.R
  code/
  data/
  output/
  docs/
  lab_diary.md
```

The `.Rproj` file opens the project with this folder as the working directory

Separate files by function

```
code/                   scripts and reusable functions
data/raw/               source files as retrieved
data/derived/           files constructed from source data
output/figures/         figures produced by code
output/tables/          tables produced by code
docs/                   codebook and project documentation
```

Raw inputs remain unchanged. Scripts rebuild derived data and outputs

Paths should travel with the project

Use paths relative to the project root

```
read.csv("data/raw/streamflow.csv")
```

A path that begins with a person's user directory usually fails on another computer

```
read.csv("C:/Users/jane/Desktop/streamflow.csv")
```

Automate the sequence

`run_all.R` expresses the order of operations

```r
source("code/00_setup.R")
source("code/01_get_cdl.R")
source("code/02_get_streamflow.R")
source("code/03_get_weather.R")
source("code/04_get_acs.R")
source("code/05_build_daily.R")
source("code/06_outputs.R")
```

One command rebuilds the project from documented inputs

Discussion: where does memory enter the workflow?

Consider one result in your current research project

- Which steps happen through code
- Which steps happen through menus or manual edits
- Where does the order of operations live
- Which inputs could change over time
- What would another researcher need to ask you before running it

What version control adds

Version control records selected states of a project and the changes connecting them

A useful history can answer

- What changed
- Who made the change
- When the change was recorded
- Why the change was made
- Which version produced a reported result

Git, GitHub, and RStudio have different roles

Tool	Role in the workflow
Git	Records file changes and commits on the computer
GitHub	Hosts a remote copy and supports sharing and collaboration
RStudio	Provides buttons and menus that run Git commands

Git supplies the version history. GitHub gives that history an online location

A repository is the project and its history

A **repository** contains

- The working files currently visible in the project folder
- A hidden `.git` directory containing the local history
- Optional connections to remote repositories such as GitHub

The project folder becomes a Git repository when Git is initialized or when a repository is cloned

Three locations in the workflow

```
Working files              Local Git history              GitHub repository
on your computer           on your computer               online remote

edit files  --commit-->    saved checkpoints  --push-->  shared checkpoints
            <--restore--                       <--pull--
```

A commit saves to the local history. A push sends local commits to GitHub

Setup checkpoint

Lecture 5.2 assigned Parts One and Two of `docs/setup-rstudio.md`

Run these checks in the RStudio Console

```
system("git --version")
usethis::git_sitrep()
nzchar(Sys.getenv("CENSUS_API_KEY"))
```

Look for a Git version, your GitHub identity and token, and `[1] TRUE`

How a repository reaches your computer

```
New research project       Create repository  --> Clone
Course demo                Fork repository    --> Clone
Lab project scaffold       Use template       --> Clone
Coauthor's repository                              Clone
```

Create, **fork**, and **use template** describe how a remote repository begins

Clone creates the local project folder and Git history

Create a repository for your research

Use **create** when the project belongs to you or your research team

For an existing RStudio project

```
usethis::use_git()
usethis::use_github(private = TRUE)
```

Review `.gitignore` before the first commit. Keep restricted or licensed data outside GitHub

Clone a repository you can access

Use **clone** when the remote repository already exists and you need a local working copy

In RStudio

1. Copy the repository's HTTPS URL from **Code**
2. Select **File > New Project > Version Control > Git**
3. Paste the URL and choose a parent folder
4. Select **Create Project**

Cloning copies files, branches, and history and configures the remote as `origin`

Fork a repository into your account

Use **fork** when you want a linked copy of another person's repository under your GitHub account

A fork supports

- Pulling later updates from the original repository
- Making changes in a repository you own
- Proposing changes to the original repository in a pull request

For this course, fork the demo so you can receive updates across Lectures 5.3 to 5.5

Use a template for an independent project

Use **template** when the files provide a useful starting structure for a new project

The new repository receives

- The template's files and folders
- A new history beginning in the new repository
- The visibility setting you choose

This is useful for a lab project layout or a standard replication package

Choose the starting action

What action fits each situation?

A. You are beginning your dissertation chapter in an empty project

B. A coauthor added you to an existing private repository

C. You want your own copy of the class demo and need later course updates

D. Your lab publishes a standard project layout for independent studies

Follow along: fork the demo

Open github.com/laurenchenarides/arec_570_demo while signed in

1. Select **Fork**
2. Uncheck **Copy the main branch only** so the lecture tags are included
3. Select **Create fork**
4. Confirm that the address is `github.com/your-username/arec_570_demo`

Your fork remains linked to the course repository for later synchronization

Follow along: clone your fork

1. On your fork, select **Code** and copy the HTTPS URL
2. In RStudio select **File > New Project > Version Control > Git**
3. Paste the URL and choose a regular parent folder such as `GitProjects`
4. Select **Create Project**
5. Confirm that the project name appears in the upper-right corner
6. Confirm that a **Git** tab appears beside Environment and History

The local repository's `origin` points to your fork

Origin and upstream describe remote locations

Name	Repository
`origin`	The remote repository cloned to your computer, which is your fork in this exercise
`upstream`	A conventional name for the original course repository

GitHub's **Sync fork** retrieves changes from the course repository into your fork

After syncing the fork, **Pull** retrieves those changes to the local project

Inspect the change from Lecture 5.2 to 5.3

Open the comparison between `5.2-retrieve` and `5.3-structure`

Look for

- The division of one retrieval script into numbered scripts
- The addition of `run_all.R`
- The separation of raw data, derived data, and outputs
- The addition of `.gitignore`
- The retrieval log and documentation

Which change most improves another researcher's ability to reproduce the project?

Git records changes through three states

State	Meaning
Working directory	Files currently being edited
Staging area	Changes selected for the next commit
Commit	A recorded snapshot with an explanatory message

Staging lets one work session become several coherent commits

Read the RStudio Git pane

Status	Meaning
`M`	A tracked file was modified
`?`	A new file is untracked
`A`	A new file is staged for its first commit
`D`	A tracked file was deleted
`R`	A tracked file was renamed or moved

Diff shows changed lines. The **Staged** boxes select files for the next commit

The everyday Git loop

1. **Pull** before beginning a work session
2. Make one coherent change
3. Inspect the changed lines with **Diff**
4. Stage the files that belong to the change
5. Write a specific commit message and select **Commit**
6. **Push** the commits to GitHub

Repeat the edit, inspect, stage, and commit steps as the work develops

Follow along: record one research decision

Create `docs/class-notes.md` and add one sentence explaining a geographic or temporal choice in the demo project

Then use the Git pane

1. Inspect the changed line with **Diff**
2. Check the **Staged** box for `docs/class-notes.md`
3. Commit with `Document geographic assignment rule`
4. Select **Push**
5. Open your fork on GitHub and locate the new commit

The corresponding Git commands

```
git pull
git status
git diff
git add docs/class-notes.md
git diff --staged
git commit -m "Document geographic assignment rule"
git push
```

The RStudio buttons run these commands

Companion reference: `docs/git-in-rstudio.md`

Write commit messages for future readers

General message	Research-specific message
`updates`	`Code suppressed ACS estimates as missing`
`fixed bug`	`Keep FIPS codes as text to preserve leading zeros`
`final version`	`Restrict sample to the April-September growing season`

A useful message identifies the completed change. Add an explanation when a change alters the sample or interpretation

Pull and push move commits

Pull brings remote commits into the local repository

Push sends local commits to the remote repository

When a push is rejected because the remote has newer commits

1. Pull the remote commits
2. Review any conflicts Git reports
3. Complete the merge if required
4. Push the combined history

Lecture 5.4 develops branches, conflicts, and pull requests

History makes earlier states inspectable

Use **History** in RStudio or **Commits** on GitHub to inspect

- The files changed in a commit
- The exact lines added and removed
- The author and date
- The commit message
- The identifier for the snapshot

Tags such as `5.2-retrieve` give a name to a particular commit

What belongs in the repository

Track files that explain or implement the research

- Source code
- README files and project documentation
- Data dictionaries and retrieval logs
- Small configuration files without credentials
- Manuscript and presentation source files

Commit generated outputs when the research team or publication process requires them

What belongs outside the repository

Common exclusions in `.gitignore`

- API keys and credentials such as `.Renviron`
- Confidential or restricted data
- Large source files stored in an archive
- Derived files rebuilt by code
- R session files such as `.RData` and `.Rhistory`

`.gitignore` applies to files Git has not started tracking

Check before the first commit

Open `.gitignore` and `git status` before staging a new project

Ask

- Does any file contain a password or API key
- Does the project contain restricted or licensed data
- Is a large file available from a stable archive or reproducible download
- Can a derived file be rebuilt by the project scripts
- Does each tracked file help explain, run, or report the research

Revoke any credential that enters a commit because earlier commits retain the file contents

Common problems and first checks

Problem	First check
The Git tab is missing	Confirm that the `.Rproj` inside a Git repository is open
Git rejects a commit	Check the configured name and email with `usethis::git_sitrep()`
GitHub rejects authentication	Check whether the personal access token is present and current
A push is rejected	Pull and inspect the incoming commits
A file never appears in the Git pane	Check the patterns in `.gitignore`

Record the exact error message when asking for help

Apply the workflow to your research project

For one current project, identify

1. The project root
2. The command or script that reproduces the main result
3. The files Git should track
4. The files `.gitignore` should exclude
5. The people who need repository access
6. The first research decision that deserves a commit

Discuss one unresolved choice with the class

Return to Gentzkow and Shapiro

How do the reading's recommendations appear in the demo repository?

- Automation through `run_all.R`
- Separate directories for code, raw data, derived data, and outputs
- Relative paths from the project root
- Version history for code and documentation
- README files that describe execution and file roles

Which recommendation would require the largest change in your current workflow?

Before you leave

Confirm that your fork contains one new commit from today's exercise

Be prepared to show

- The line you changed
- The commit message
- The same commit in RStudio and on GitHub
- One file your research repository should exclude

Summary

Project structure records where inputs, code, and outputs belong

Git records local changes as commits

GitHub hosts a remote repository for sharing and collaboration

Create, fork, and template determine how a repository begins

Clone creates the local working copy

Pull, edit, stage, commit, and push form the everyday workflow

References

Gentzkow, Matthew, and Jesse M. Shapiro. 2014. *Code and Data for the Social Sciences: A Practitioner's Guide*. University of Chicago mimeo. https://web.stanford.edu/~gentzkow/research/CodeAndData.pdf.

Course navigation

- All lectures
- Schedule and preparation
- Course reading list
- Download this lecture as a PDF